

NOTE : PLEASE STUDY AT YOUR RISK, WE NOT RESPONSIBLE FOR INCIDENT OR CHANGE OF WORDS OR CHANGE IN ACTION OR CHANGE IN PROGRAM BY (IKYK)

PLEASE UNDERSTAND AND ACT !!

ALL PROGRAMS ARE TESTED AND WORKED PERFECTLY

USE ONLY : PYTHON 3.11.9 [I tested] [uninstall any other version]

INSTALL PACKAGES :

python -m pip install --upgrade pip

pip install tensorflow numpy== matplotlib

WHILE RUNNING PACKAGE ERROR MESSAGE MAY APPEAR IGNORE THAT <3

1) A) & 9) A) Implement simple vector addition in TensorFlow

Procedure

1. Import TensorFlow.
2. Create two vectors.
3. Add the vectors using TensorFlow.
4. Display the result.

Code

```
import os

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"


import tensorflow as tf


a = tf.constant([10, 20, 30, 40])
b = tf.constant([1, 2, 3, 4])


result = tf.add(a, b)


print("Vector A:", a.numpy())
print("Vector B:", b.numpy())
print("Addition:", result.numpy())
```

1) B), 10) A), 11) A) & 20) A) Implement a regression model using Keras

Procedure

1. Import required libraries.
2. Create input and output data.
3. Build a regression model.
4. Compile and train the model.
5. Predict output for a new input.

Code

```
import os

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"
```

```

import numpy as np

import tensorflow as tf

from tensorflow.keras import layers, models

tf.keras.utils.set_random_seed(7)

x = np.array([1, 2, 3, 4, 5, 6], dtype=float)
y = np.array([5, 8, 11, 14, 17, 20], dtype=float)

model = models.Sequential([
    layers.Input(shape=(1,)),
    layers.Dense(1)
])

model.compile(optimizer=tf.keras.optimizers.Adam(0.05), loss="mse")
model.fit(x, y, epochs=200, verbose=0)

test = np.array([10.0])

prediction = model.predict(test, verbose=0)

print("Input:", test[0])

print("Predicted Output:", prediction[0][0])

```

2) A), 10) B) & 11) B) Implement a perceptron using TensorFlow/Keras

Procedure

1. Create OR gate input and output data.
2. Build a single-layer perceptron.
3. Compile the model.
4. Train the model.
5. Predict OR gate output.

Code

```

import os

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"

```

```
import numpy as np

import tensorflow as tf

from tensorflow.keras import layers, models

tf.keras.utils.set_random_seed(7)

x = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]], dtype=float)

y = np.array([[0],
              [1],
              [1],
              [1]], dtype=float)

model = models.Sequential([
    layers.Input(shape=(2,)),
    layers.Dense(1, activation="sigmoid")
])

model.compile(optimizer=tf.keras.optimizers.Adam(0.1),
              loss="binary_crossentropy",
              metrics=["accuracy"])

model.fit(x, y, epochs=200, verbose=0)

pred = model.predict(x, verbose=0)

print("OR Gate using Perceptron")

for i in range(len(x)):
    print(x[i], "=>", round(pred[i][0]))
```

2) B), 12) A), 17) B) & 18) A) Implement a Feed-Forward Neural Network using TensorFlow/Keras

Procedure

1. Create XOR gate dataset.
2. Build a feed-forward neural network.
3. Add hidden layers.
4. Train the model.
5. Predict XOR output.

Code

```
import os

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"

import numpy as np
import tensorflow as tf

from tensorflow.keras import layers, models

tf.keras.utils.set_random_seed(7)

x = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]], dtype=float)

y = np.array([[0],
              [1],
              [1],
              [0]], dtype=float)

model = models.Sequential([
    layers.Input(shape=(2,)),
    layers.Dense(4, activation="tanh"),
    layers.Dense(4, activation="tanh"),
```

```

layers.Dense(1, activation="sigmoid")
])

model.compile(optimizer=tf.keras.optimizers.Adam(0.1),
              loss="binary_crossentropy",
              metrics=["accuracy"])

model.fit(x, y, epochs=800, verbose=0)

pred = model.predict(x, verbose=0)

print("XOR Gate using Feed-Forward Network")
for i in range(len(x)):
    print(x[i], "=>", round(pred[i][0]))

```

3) A) & 13) A) Implement CNN-based Image Classification

Procedure

1. Create simple image data.
2. Use vertical line as class 0.
3. Use horizontal line as class 1.
4. Build a CNN model.
5. Train and test the model.

Code

```

import os

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"

import numpy as np
import tensorflow as tf

from tensorflow.keras import layers, models

tf.keras.utils.set_random_seed(7)

```

```
def create_images(samples=400):  
    x = np.zeros((samples, 16, 16, 1))  
    y = np.zeros(samples)  
    for i in range(samples):  
        if i % 2 == 0:  
            x[i, :, 7:9, 0] = 1.0  
            y[i] = 0  
        else:  
            x[i, 7:9, :, 0] = 1.0  
            y[i] = 1  
        x[i] += np.random.normal(0, 0.05, x[i].shape)  
    return x, y  
x, y = create_images()
```

```
x_train, y_train = x[:300], y[:300]  
x_test, y_test = x[300:], y[300:]
```

```
model = models.Sequential([  
    layers.Input(shape=(16, 16, 1)),  
    layers.Conv2D(8, (3, 3), activation="relu"),  
    layers.MaxPooling2D((2, 2)),  
    layers.Flatten(),  
    layers.Dense(16, activation="relu"),  
    layers.Dense(2, activation="softmax")  
])
```

```
model.compile(optimizer="adam",  
              loss="sparse_categorical_crossentropy",  
              metrics=["accuracy"])
```

```
model.fit(x_train, y_train, epochs=5, verbose=0)
```

```
loss, accuracy = model.evaluate(x_test, y_test, verbose=0)
prediction = model.predict(x_test[:1], verbose=0)
print("Test Accuracy:", accuracy)
print("Predicted Class:", np.argmax(prediction))
print("Actual Class:", int(y_test[0]))
```

3) B), 13) B) & 20) B) Improve a Deep Learning model by fine-tuning hyperparameters

Procedure

1. Create image classification data.
2. Build a CNN model function.
3. Change filters and learning rate.
4. Train different models.
5. Compare accuracy.

Code

```
import os

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"


import numpy as np
import tensorflow as tf

from tensorflow.keras import layers, models

tf.keras.utils.set_random_seed(7)


def create_images(samples=400):
    x = np.zeros((samples, 16, 16, 1))
    y = np.zeros(samples)

    for i in range(samples):
        if i % 2 == 0:
            x[i, :, 7:9, 0] = 1.0
            y[i] = 0
```


else:

 x[i, 7:9, :, 0] = 1.0

 y[i] = 1

 x[i] += np.random.normal(0, 0.05, x[i].shape)

return x, y

def build_model(filters, learning_rate):

 model = models.Sequential([

 layers.Input(shape=(16, 16, 1)),

 layers.Conv2D(filters, (3, 3), activation="relu"),

 layers.MaxPooling2D((2, 2)),

 layers.Flatten(),

 layers.Dense(16, activation="relu"),

 layers.Dense(2, activation="softmax")

])

 model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate),

 loss="sparse_categorical_crossentropy",

 metrics=["accuracy"])

 return model

x, y = create_images()

x_train, y_train = x[:300], y[:300]

x_test, y_test = x[300:], y[300:]

settings = [

 [4, 0.001],

 [8, 0.001],

 [8, 0.0005]

]

for filters, lr in settings:

```
model = build_model(filters, lr)
```

```
model.fit(x_train, y_train, epochs=3, verbose=0)
```

```
loss, accuracy = model.evaluate(x_test, y_test, verbose=0)
```

```
print("Filters:", filters, "Learning Rate:", lr, "Accuracy:", accuracy)
```

4) A), 8) B), 16) B) & 19) A) Implement Transfer Learning for Image Classification

Procedure

1. Create simple image data.
2. Train a base CNN model.
3. Freeze the base model.
4. Add new classification layers.
5. Train only the new classifier.

Code

```
import os
```

```
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"
```

```
import numpy as np
```

```
import tensorflow as tf
```

```
from tensorflow.keras import layers, models
```

```
tf.keras.utils.set_random_seed(7)
```

```
def create_images(samples=400):
```

```
    x = np.zeros((samples, 16, 16, 1))
```

```
    y = np.zeros(samples)
```

```
    for i in range(samples):
```

```
        if i % 2 == 0:
```

```
x[i, :, 7:9, 0] = 1.0
```

```
y[i] = 0
```

```
else:
```

```
x[i, 7:9, :, 0] = 1.0
```

```
y[i] = 1
```

```
x[i] += np.random.normal(0, 0.05, x[i].shape)
```

```
return x, y
```

```
x, y = create_images()
```

```
x_train, y_train = x[:300], y[:300]
```

```
x_test, y_test = x[300:], y[300:]
```

```
base_model = models.Sequential([  
    layers.Input(shape=(16, 16, 1)),  
    layers.Conv2D(8, (3, 3), activation="relu"),  
    layers.MaxPooling2D((2, 2)),  
    layers.Flatten()  
])
```

```
pretrain_model = models.Sequential([  
    base_model,  
    layers.Dense(16, activation="relu"),  
    layers.Dense(2, activation="softmax")  
])
```

```
pretrain_model.compile(optimizer="adam",  
                        loss="sparse_categorical_crossentropy",  
                        metrics=["accuracy"])
```

```
pretrain_model.fit(x_train, y_train, epochs=3, verbose=0)
```

```
base_model.trainable = False
```

```
transfer_model = models.Sequential([  
    base_model,  
    layers.Dense(16, activation="relu"),  
    layers.Dense(2, activation="softmax")  
])
```

```
transfer_model.compile(optimizer="adam",  
                       loss="sparse_categorical_crossentropy",  
                       metrics=["accuracy"])
```

```
transfer_model.fit(x_train, y_train, epochs=3, verbose=0)
```

```
loss, accuracy = transfer_model.evaluate(x_test, y_test, verbose=0)
```

```
print("Transfer Learning Accuracy:", accuracy)
```

4) B) & 14) A) Use a pre-trained model in Keras for Transfer Learning

Procedure

1. Create image data.
2. Train and save a small Keras model.
3. Load the saved model as a pre-trained model.
4. Freeze some layers.
5. Train and test the model.

Code

```
import os

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"


import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models


tf.keras.utils.set_random_seed(7)


def create_images(samples=400):
    x = np.zeros((samples, 16, 16, 1))
    y = np.zeros(samples)

    for i in range(samples):
        if i % 2 == 0:
            x[i, :, 7:9, 0] = 1.0
            y[i] = 0
        else:
            x[i, 7:9, :, 0] = 1.0
            y[i] = 1

    x[i] += np.random.normal(0, 0.05, x[i].shape)

    return x, y


x, y = create_images()


x_train, y_train = x[:300], y[:300]
x_test, y_test = x[300:], y[300:]
```

```
model = models.Sequential([
    layers.Input(shape=(16, 16, 1)),
    layers.Conv2D(8, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(16, activation="relu"),
    layers.Dense(2, activation="softmax")
])
```

```
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
```

```
model.fit(x_train, y_train, epochs=3, verbose=0)
```

```
model.save("pretrained_light_model.keras")
```

```
pretrained_model = tf.keras.models.load_model("pretrained_light_model.keras")
```

```
for layer in pretrained_model.layers[:-1]:
```

```
    layer.trainable = False
```

```
pretrained_model.compile(optimizer="adam",
                        loss="sparse_categorical_crossentropy",
                        metrics=["accuracy"])
```

```
pretrained_model.fit(x_train, y_train, epochs=2, verbose=0)
```

```
loss, accuracy = pretrained_model.evaluate(x_test, y_test, verbose=0)
```

```
print("Accuracy using Pre-trained Keras Model:", accuracy)
```

5) A), 15) A) & 19) B) Perform Sentiment Analysis using RNN

Procedure

1. Create positive and negative text data.
2. Convert text into number sequences.
3. Pad the sequences.
4. Build an RNN model.
5. Predict sentiment.

Code

```
import os

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"


import numpy as np
import tensorflow as tf

from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

tf.keras.utils.set_random_seed(7)

texts = [
    "good movie", "nice film", "happy story", "excellent acting",
    "bad movie", "boring film", "sad story", "poor acting"
]

labels = np.array([1, 1, 1, 1, 0, 0, 0, 0])

tokenizer = Tokenizer(num_words=50)
tokenizer.fit_on_texts(texts)

sequences = tokenizer.texts_to_sequences(texts)
x = pad_sequences(sequences, maxlen=3)
```

```

model = models.Sequential([
    layers.Input(shape=(3,)),
    layers.Embedding(input_dim=50, output_dim=8),
    layers.SimpleRNN(8),
    layers.Dense(1, activation="sigmoid")
])

model.compile(optimizer="adam",
              loss="binary_crossentropy",
              metrics=["accuracy"])

model.fit(x, labels, epochs=80, verbose=0)

test_text = ["good acting"]
test_sequence = tokenizer.texts_to_sequences(test_text)
test_padded = pad_sequences(test_sequence, maxlen=3)

prediction = model.predict(test_padded, verbose=0)

print("Sentence:", test_text[0])
print("Sentiment:", "Positive" if prediction[0][0] > 0.5 else "Negative")

```

5) B), 12) B) & 18) B) Implement an LSTM-based Autoencoder in TensorFlow/Keras

Procedure

1. Create simple sequence data.
2. Build LSTM encoder.
3. Build LSTM decoder.
4. Train model to reconstruct input.
5. Print original and reconstructed values.

Code


```
import os

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"


import numpy as np

import tensorflow as tf

from tensorflow.keras import layers, models


tf.keras.utils.set_random_seed(7)


samples = 300

timesteps = 10


x = []


for i in range(samples):

    sequence = np.sin(np.linspace(0, 3.14, timesteps) + np.random.rand())

    x.append(sequence)


x = np.array(x).reshape(samples, timesteps, 1)


model = models.Sequential([

    layers.Input(shape=(timesteps, 1)),

    layers.LSTM(8),

    layers.RepeatVector(timesteps),

    layers.LSTM(8, return_sequences=True),

    layers.TimeDistributed(layers.Dense(1))

])


model.compile(optimizer="adam", loss="mse")


model.fit(x, x, epochs=5, batch_size=16, verbose=0)
```

```
reconstructed = model.predict(x[:1], verbose=0)
```

```
print("Original First 5 Values:")
```

```
print(x[0].flatten()[:5])
```

```
print("Reconstructed First 5 Values:")
```

```
print(reconstructed[0].flatten()[:5])
```

6) A) & 14) B) Train a Deep Learning model using a pre-trained model for image classification

Procedure

1. Create image classification data.
2. Train a small CNN model.
3. Save it as a pre-trained model.
4. Load the saved model.
5. Classify a test image.

Code

```
import os
```

```
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"
```

```
import numpy as np
```

```
import tensorflow as tf
```

```
from tensorflow.keras import layers, models
```

```
tf.keras.utils.set_random_seed(7)
```

```
def create_images(samples=400):
```

```
    x = np.zeros((samples, 16, 16, 1))
```

```
    y = np.zeros(samples)
```

```
    for i in range(samples):
```

```

if i % 2 == 0:
    x[i, :, 7:9, 0] = 1.0
    y[i] = 0
else:
    x[i, 7:9, :, 0] = 1.0
    y[i] = 1

x[i] += np.random.normal(0, 0.05, x[i].shape)

return x, y

x, y = create_images()

x_train, y_train = x[:300], y[:300]
x_test, y_test = x[300:], y[300:]

model = models.Sequential([
    layers.Input(shape=(16, 16, 1)),
    layers.Conv2D(8, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(16, activation="relu"),
    layers.Dense(2, activation="softmax")
])

model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

model.fit(x_train, y_train, epochs=3, verbose=0)

```

```
model.save("image_classifier_model.keras")
```

```
pretrained_model = tf.keras.models.load_model("image_classifier_model.keras")
```

```
prediction = pretrained_model.predict(x_test[:1], verbose=0)
```

```
print("Predicted Class:", np.argmax(prediction))
```

```
print("Actual Class:", int(y_test[0]))
```

6) B) & 9) B) Implement Object Detection using CNN

Procedure

1. Create small images with a white square object.
2. Store object location as bounding box.
3. Build a CNN model.
4. Train the model.
5. Predict bounding box values.

Code

```
import os
```

```
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"
```

```
import numpy as np
```

```
import tensorflow as tf
```

```
from tensorflow.keras import layers, models
```

```
tf.keras.utils.set_random_seed(7)
```

```
def create_data(samples=500):
```

```
    images = []
```

```
    boxes = []
```

```
    for i in range(samples):
```

```
image = np.zeros((16, 16, 1))
```

```
x = np.random.randint(2, 10)
```

```
y = np.random.randint(2, 10)
```

```
size = 4
```

```
image[y:y+size, x:x+size, 0] = 1.0
```

```
box = [  
    x / 16,  
    y / 16,  
    (x + size) / 16,  
    (y + size) / 16  
]
```

```
images.append(image)
```

```
boxes.append(box)
```

```
return np.array(images), np.array(boxes)
```

```
x, y = create_data()
```

```
x_train, y_train = x[:400], y[:400]
```

```
x_test, y_test = x[400:], y[400:]
```

```
model = models.Sequential([  
    layers.Input(shape=(16, 16, 1)),  
    layers.Conv2D(8, (3, 3), activation="relu"),  
    layers.MaxPooling2D((2, 2)),  
    layers.Flatten(),  
    layers.Dense(16, activation="relu"),  
])
```

```

layers.Dense(4, activation="sigmoid")
])

model.compile(optimizer="adam", loss="mse")

model.fit(x_train, y_train, epochs=5, verbose=0)

prediction = model.predict(x_test[:1], verbose=0)

print("Actual Bounding Box:", y_test[0])
print("Predicted Bounding Box:", prediction[0])

```

7) A) & 17) A) Implement a simple Reinforcement Learning algorithm for an NLP problem

Procedure

1. Create positive and negative sentences.
2. Convert sentence into state.
3. Define actions as positive or negative.
4. Train Q-table using rewards.
5. Predict sentiment for test sentences.

Code

```

import numpy as np
import random

texts = [
    "good movie", "nice film", "happy story", "excellent acting",
    "bad movie", "boring film", "sad story", "poor acting"
]

labels = [1, 1, 1, 1, 0, 0, 0, 0]

positive_words = ["good", "nice", "happy", "excellent"]

```

```
negative_words = ["bad", "boring", "sad", "poor"]
```

```
def get_state(sentence):
```

```
    words = sentence.split()
```

```
    for word in words:
```

```
        if word in positive_words:
```

```
            return 0
```

```
    for word in words:
```

```
        if word in negative_words:
```

```
            return 1
```

```
    return 2
```

```
states = 3
```

```
actions = 2
```

```
q_table = np.zeros((states, actions))
```

```
alpha = 0.1
```

```
epsilon = 0.2
```

```
for episode in range(500):
```

```
    index = random.randint(0, len(texts) - 1)
```

```
    sentence = texts[index]
```

```
    correct = labels[index]
```

```
    state = get_state(sentence)
```

```

if random.random() < epsilon:
    action = random.randint(0, 1)
else:
    action = np.argmax(q_table[state])

reward = 1 if action == correct else -1

q_table[state, action] = q_table[state, action] + alpha * (
    reward - q_table[state, action]
)

print("Trained Q-Table:")
print(q_table)

test_sentences = ["good story", "boring movie", "happy film", "bad acting"]

for sentence in test_sentences:
    state = get_state(sentence)
    action = np.argmax(q_table[state])

    result = "Positive" if action == 1 else "Negative"

    print(sentence, "=>", result)

```

7) B) & 15) B) Build a Recommendation System using Deep Learning

Procedure

1. Create user, product, and rating data.
2. Use embedding layers.
3. Combine user and product embeddings.
4. Train the model.
5. Recommend product for a user.

Code

```
import os

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"


import numpy as np

import tensorflow as tf

from tensorflow.keras import layers, models


tf.keras.utils.set_random_seed(7)


num_users = 5

num_products = 6


users = []

products = []

ratings = []


for u in range(num_users):
    for p in range(num_products):
        users.append(u)
        products.append(p)

        rating = ((u + 1) * (p + 2)) % 5 + 1
        ratings.append(rating)


users = np.array(users)

products = np.array(products)

ratings = np.array(ratings, dtype=float) / 5.0


user_input = layers.Input(shape=(1,))

product_input = layers.Input(shape=(1,))
```

```

user_embedding = layers.Embedding(num_users, 4)(user_input)
product_embedding = layers.Embedding(num_products, 4)(product_input)

dot_product = layers.Dot(axes=2)([user_embedding, product_embedding])
output = layers.Flatten()(dot_product)

model = models.Model([user_input, product_input], output)

model.compile(optimizer="adam", loss="mse")

model.fit([users, products], ratings, epochs=100, verbose=0)

test_user = 2

test_users = np.array([test_user] * num_products)
test_products = np.array(range(num_products))

prediction = model.predict([test_users, test_products], verbose=0)

recommended_product = np.argmax(prediction)

print("User ID:", test_user)
print("Predicted Ratings:", prediction.flatten())
print("Recommended Product ID:", recommended_product)

```

8) A) & 16) A) Implement GAN for Image Generation

Procedure

1. Create simple image data.
2. Build a generator model.
3. Build a discriminator model.

4. Train both models.
5. Generate a new image.

Code

```
import os

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"


import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models

tf.keras.utils.set_random_seed(7)


def create_real_images(samples=200):
    images = []

    for i in range(samples):
        image = np.zeros((8, 8))

        if i % 2 == 0:
            image[:, 3:5] = 1.0
        else:
            image[3:5, :] = 1.0

        images.append(image.flatten())

    return np.array(images)

real_images = create_real_images()

latent_dim = 10
```

```
generator = models.Sequential([
    layers.Input(shape=(latent_dim,)),
    layers.Dense(16, activation="relu"),
    layers.Dense(64, activation="sigmoid")
])
```

```
discriminator = models.Sequential([
    layers.Input(shape=(64,)),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])
```

```
loss_function = tf.keras.losses.BinaryCrossentropy()
generator_optimizer = tf.keras.optimizers.Adam(0.01)
discriminator_optimizer = tf.keras.optimizers.Adam(0.01)
```

```
for step in range(100):
    index = np.random.randint(0, len(real_images), 16)
    real = real_images[index]

    noise = tf.random.normal((16, latent_dim))
    fake = generator(noise)

    with tf.GradientTape() as tape:
        real_output = discriminator(real)
        fake_output = discriminator(fake)

    discriminator_loss = loss_function(tf.ones((16, 1)), real_output) + \
        loss_function(tf.zeros((16, 1)), fake_output)

    gradients = tape.gradient(discriminator_loss,
```

```
discriminator.trainable_variables)
```

```
discriminator_optimizer.apply_gradients(  
    zip(gradients, discriminator.trainable_variables)  
)
```

```
noise = tf.random.normal((16, latent_dim))
```

```
with tf.GradientTape() as tape:
```

```
    generated = generator(noise)
```

```
    output = discriminator(generated)
```

```
    generator_loss = loss_function(tf.ones((16, 1)), output)
```

```
gradients = tape.gradient(generator_loss,  
                           generator.trainable_variables)
```

```
generator_optimizer.apply_gradients(  
    zip(gradients, generator.trainable_variables)  
)
```

```
print("GAN Training Completed")
```

```
noise = tf.random.normal((1, latent_dim))
```

```
new_image = generator(noise).numpy().reshape(8, 8)
```

```
print("Generated 8x8 Image:")
```

```
print(np.round(new_image, 2))
```